不易流行の Agentic Coding

薩摩の偉人と学ぶ、ソフトウェアエンジニアリング





目次

導入 — 松尾芭蕉

01 決める — 西郷隆盛

02 組む — 大久保利通

03 確かめる — 島津忠良（日新公）

04 捨てる — 西郷隆盛

締め 今日の4つ

“ 不易を知らざれば基立ちがたく、
流行を知らざれば風新たならず

（訳）変わらない基本を知らなければ土台が立たず、変わっていくものを知らなければ新しさは生まれない

— 松尾芭蕉
芭蕉の教え（向井去来『去来抄』）



画像：Wikimedia Commons（パブリックドメイン）

不易流行とは



変わらないもの 不易

いつの時代も変わらない、
物事の本質

×



変わるもの 流行

時代とともに、
新しさを求めて変わっていくもの

“

その基は一つなり

— 芭蕉の教え 向井去来『去来抄』

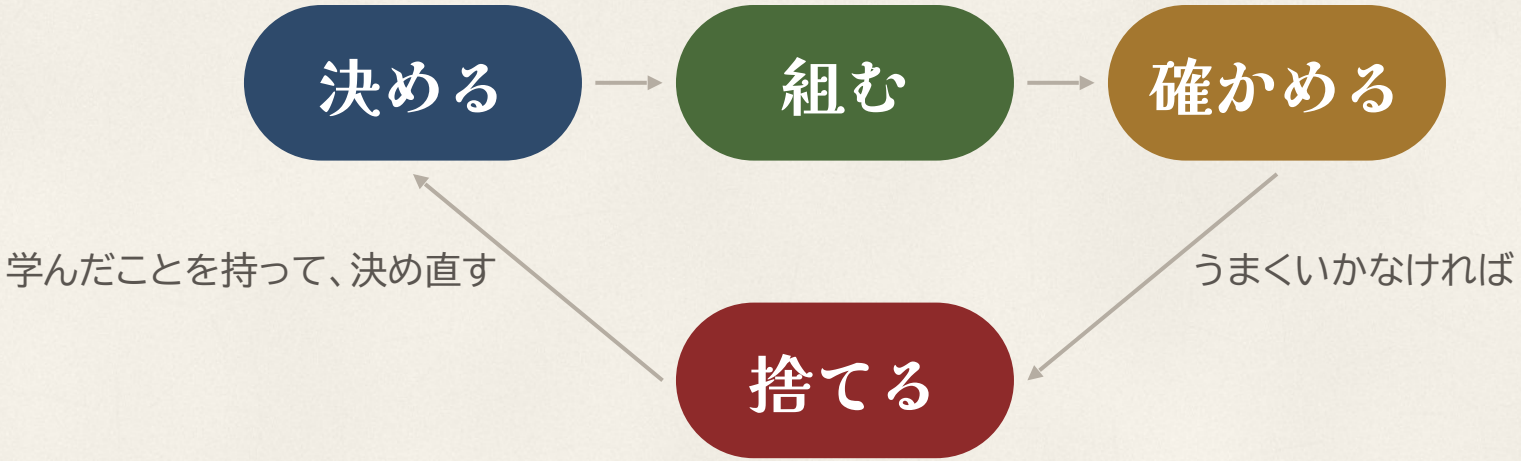
今日の読み替え　不易＝ソフトウェアエンジニアリングの基礎／流行＝AIとAgentic Coding

AIは増幅器 “AI is an amplifier”

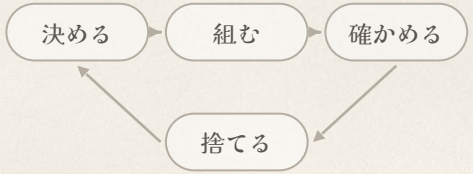
変わらない	変わる
何を作るかは人が決める	コードの多くをAIに書かせられる
要件を言葉にする力が要る	要件の書き方が、そのままAIへの指示になる
小さく作り、つなげていく	AIが一度に作れる範囲が大きく広がる
確かめなければ信用できない	人の役割が「書く」から「決める・確かめる」へ移る
捨てる判断と作り直しの検証は人がする	コードを再生成する手間が大きく下がる

> 強い組織の強みも、困っている組織の問題も大きくする
>
> — DORA『ROI of AI-assisted Software Development』2026

人に残る4つの仕事



> 難しいのは、仕様にし、設計し、テストすることで、書き表す作業ではない
>
> — Fred Brooks “No Silver Bullet” 1986



このあと実例として出てくる：Senrigan

SENRIGAN
千里眼

AWS CloudTrail の脅威ハンティングツール

組み込みのハント

120+

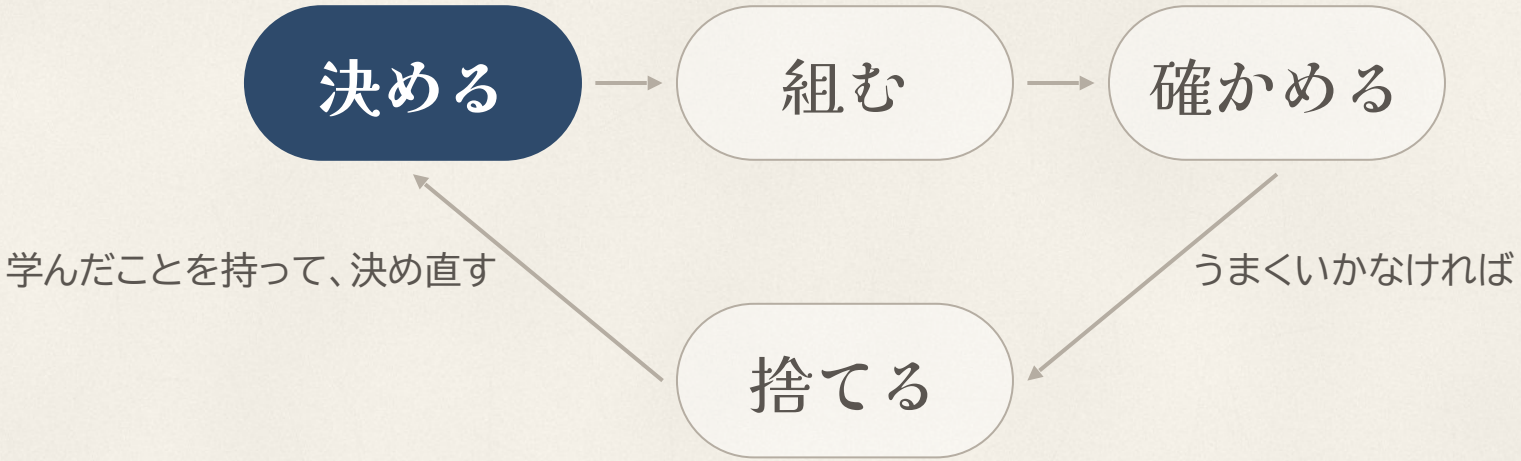
ダッシュボードのグラフ

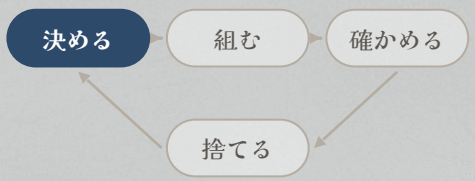
100+

[1] github.com/Yamato-Security/senrigan

出典：Senrigan の README（2026-10-29 時点）[1]

人に残る4つの仕事





“ 何程制度方法を論ずるとも、

其の人に非ざれば行はれ難し。（中略）

己れ其の人に成るの心懸け肝要なり

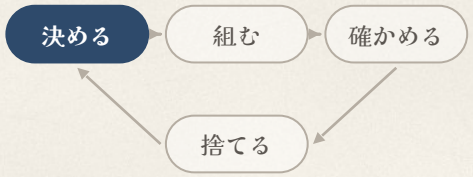
（訳）どれほど制度や方法を論じても、それを行う人がいなければ
うまくいかない。自分がその人になろうと心がけることが大切だ

— 西郷隆盛

『南洲翁遺訓』第20条（底本：角川ソフィア文庫『新版 南洲翁遺訓』）

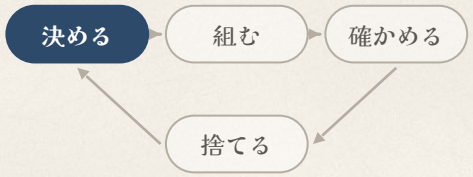


画像：Wikimedia Commons（パブリックドメイン）



決 鉄則　作る前に3行で書く

1. 誰の、何の困りごとか
2. 何ができたら完成か
3. やらないこと



不易と流行

変わらない 不易

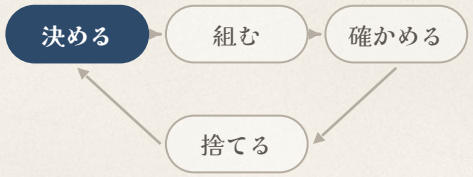
要件を言葉にする力が要る

×

変わる 流行

要件の書き方が、
そのままAIへの指示になる

書かなかった条件は、正しく伝わったか確かめられない





不易

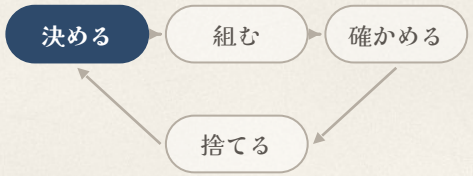
何を作るかを決めるのが、一番難しい

> ソフトウェアを作るうえで一番難しいのは、何を作るかを正確に決めることだ。
> ここを間違えるほどシステムをダメにする部分はなく、
> あとから直すのがこれほど難しい部分もない
>
> — Fred Brooks “No Silver Bullet” 1986[1]

ユーザーに向き合うことは、AIで成果を出すための前提になる

DORA 2025 調査回答[2]

[1] worrydream.com/refs/Brooks 1986 - No Silver Bullet.pdf
[2] research.google/pubs/dora-2025-state-of-ai-assisted-software-development-report/



流行

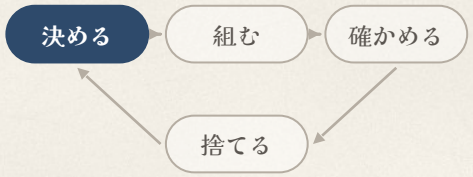
書いた要件が、そのままAIへの指示になる

> 仕様が書けるまで私にインタビューして、SPEC.md に書いて
>
> — Claude Code 公式ガイド[1]

AIが仕様からコードを作るなら、誤りを止めるのに一番効くのは仕様。
悪い仕様は、悪いコードを大量に生む

Thoughtworks “The Future of Software Engineering” 2026[2]

[1] code.claude.com/docs/en/best-practices
[2] www.thoughtworks.com/content/dam/thoughtworks/documents/report/tw future%20 of software development retreat %20key takeaways.pdf

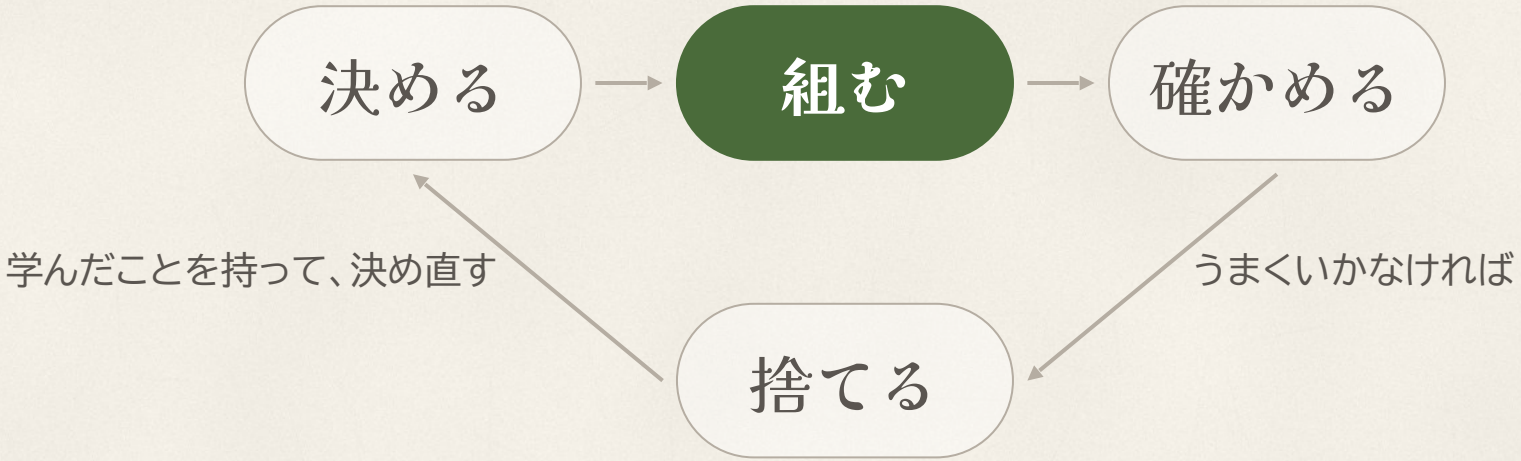


実例

決めて頼んだとき、決めずに頼んだとき

	Senrigan 本体	VPC Flow Logs の機能
決め方	AIと議論して決め、PRDにした	PRDはあったが、本物のデータでの確認は見送った
完成の条件	数字で決めた（例：10GBを5分以内に取り込む）	本物のログで何が見えたら完成か、決めていなかった
結果	**大量のログ取り込み、可視化を達成!**	**本物のログで、見たいものが見えなかった**

人に残る4つの仕事



其既ニ開成スルモノハ之ヲ保持シ、未ダ就緒ナラザルモノハ之ヲ誘導

（訳）すでにできているものは生かし、まだ整っていないものは導く

— 大久保利通

「殖産興業ニ関スル建議書」（1874年、『大久保利通文書』所収）

廃藩置県　岩倉使節団　内務省　地租改正　殖産興業

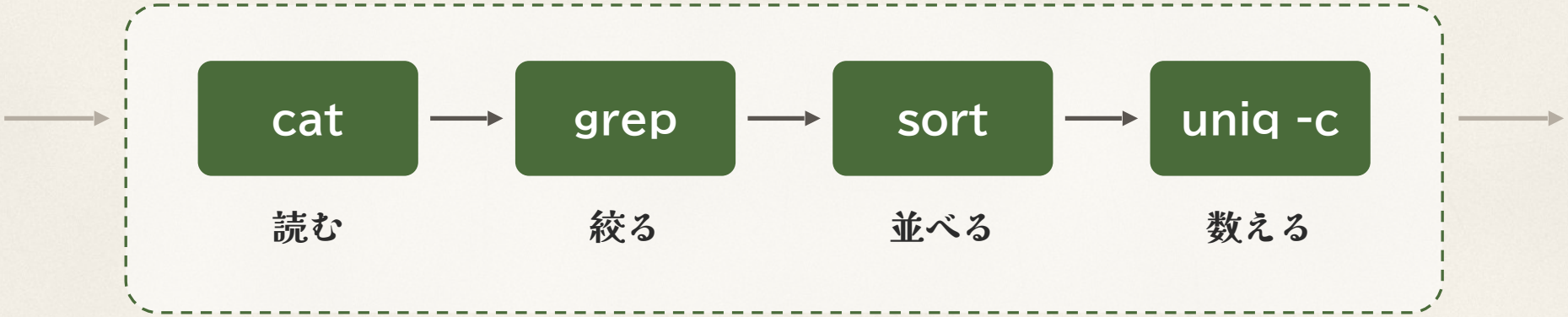


画像：Wikimedia Commons（パブリックドメイン）



組 鉄則　1つの機能は、1つのことをうまくやる

小さな部品に分け、組み合わせる



ログから、失敗したログインを数える



不易と流行

変わらない 不易

小さく作り、つなげていく

×

変わる 流行

AIが一度に作れる範囲が
大きく広がる

> AIエージェントを監督する仕事には、
> 問題をエージェント向けの大きさの作業に分ける力が要る
>
> — Thoughtworks “The Future of Software Engineering” 2026[1]

[1] www.thoughtworks.com/content/dam/thoughtworks/documents/report/tw future%20 of software development retreat %20key takeaways.pdf





不易

小さな部品を、組み合わせる

> 1つのプログラムには、1つのことをうまくやらせよ
>
> — UNIXの格言 Bell System Technical Journal 1978[1]

> 「小さな単位で進める」は、AIの効果を左右する7つの土台の1つ
>
> DORA AI Capabilities Model 2025[2]

[1] archive.org/download/bstj57-6-1899/bstj57-6-1899 text.pdf
[2] cloud.google.com/blog/products/ai-machine-learning/introducing-doras-inaugural-ai-capabilities-model





流行

AIが大きく作れる。だから小さく頼む

> システムの大きさは、しっかり握っておけ。でないと手に負えなくなる
>
> — Steve Yegge　元 Amazon・Google のエンジニア　X（旧 Twitter）への投稿 2026-09-02[1]

AIを使うほど出荷量は上がる。
ただし、不安定さ（変更の失敗、手戻り）も増える

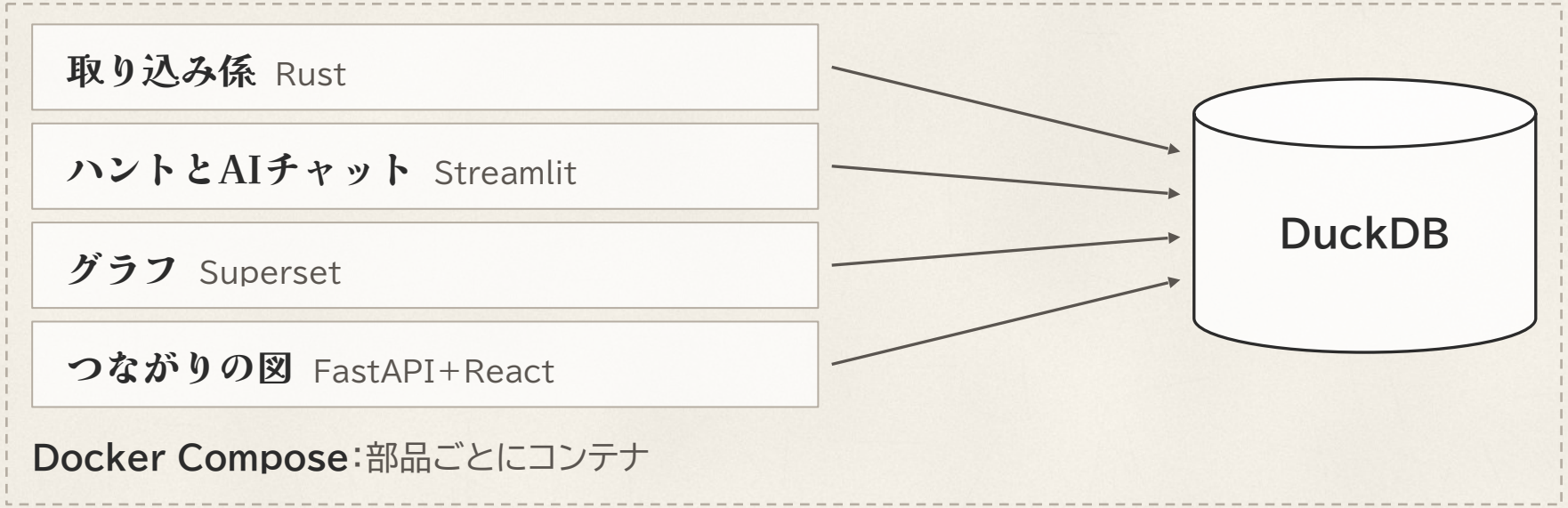
DORA 2025　調査回答[2]

[1] x.com/Steve Yegge/status/2094947586373505122
[2] research.google/pubs/dora-2025-state-of-ai-assisted-software-development-report/



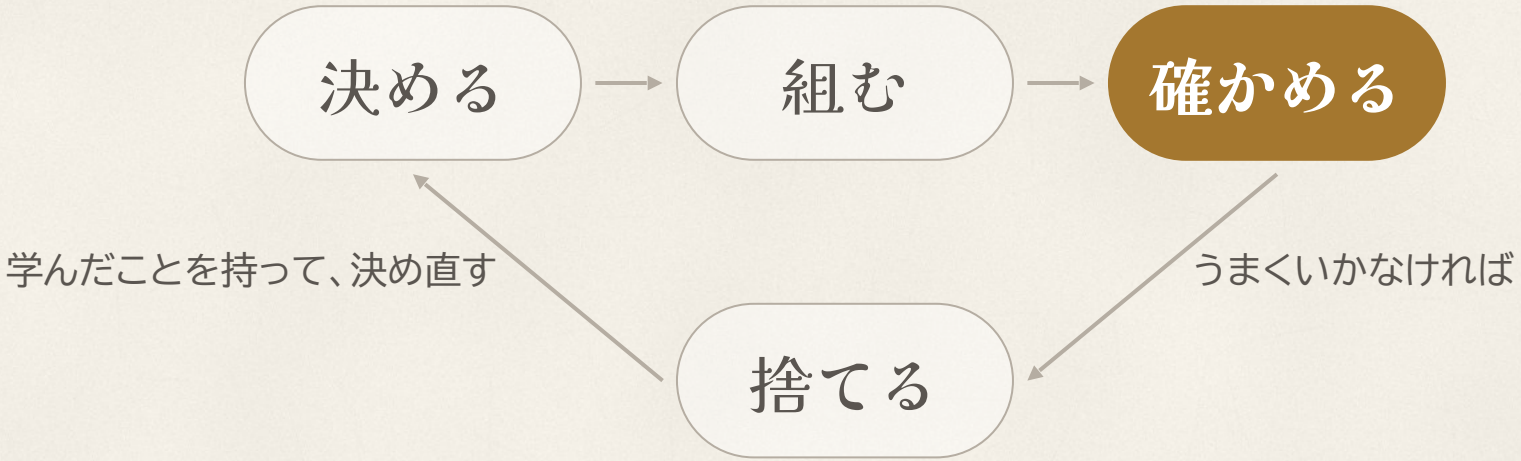
実例

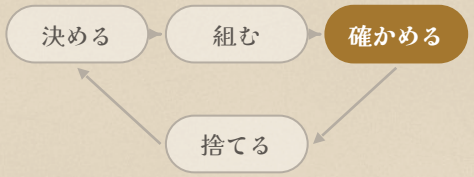
1つのことをする部品を、つなげる



Senrigan は、1つのことだけをする部品を、組み合わせて作った

人に残る4つの仕事





“ いにしへの道を聞きても唱へても
わが行ひにせずばかひなし

（訳）昔からの教えを聞いても唱えても、自分の行いにしなければ意味がない

— 島津忠良（日新公）
島津日新公いろは歌「い」



画像：Wikimedia Commons（パブリックドメイン）



確 鉄則　自分で動かして、自分の目で見る

1

データは本物で

2

「完了しました」は、コマンドと出力を見るまで信じない



不易と流行

変わらない 不易

確かめなければ信用できない

×

変わる 流行

人の役割が「書く」から
「決める・確かめる」へ移る

> 確かめる手段がなければ、AIは「完了に見える」ところで止まる。
> そうなると、あなた自身がテストスイートになる
>
> — Claude Code 公式ガイド[1]

[1] code.claude.com/docs/en/best-practices





不易

思い込みではなく、測って確かめる

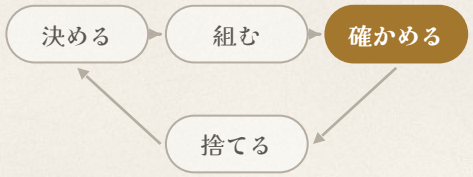
> 自動化されたテストが失敗した時のみ新しいコードを書く
>
> — Kent Beck『テスト駆動開発』2002[1]

本人の感覚
20%速くなった

実際に測った作業時間
19%長くなった

感覚と実測がずれた一例。METR 2025。経験豊富なOSS開発者16人のランダム化比較試験。2025年初めのツール[2]

[1] www.ohmsha.co.jp/book/9784274217883.html
[2] metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/



流行

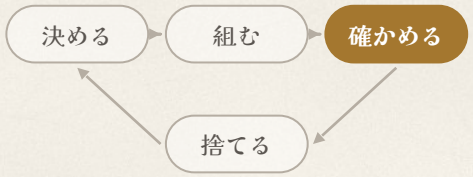
AIが書く量が増えるほど、確かめる量も増える

> テストが先にあれば、AIは、誤った実装をなぞるテストでごまかせない
>
> — Thoughtworks “The Future of Software Engineering” 2026[1]

AIの出力を信用しきれないので、レビューに時間がかかる。
さらに、コードの量が増えるので、レビューの負担そのものも増える

DORA『ROI of AI-assisted Software Development』2026[2]

[1] www.thoughtworks.com/content/dam/thoughtworks/documents/report/tw future%20 of software development retreat %20key takeaways.pdf
[2] services.google.com/fh/files/misc/dora-roi-of-ai-assisted-software-development-2026.pdf



実例

「完了しました」のあとで



S3バケット
F0 9F AA **A3**



ゴキブリ
F0 9F AA **B3**

UTF-8で、違いは**1ビット**

1010 0011 → **1011** 0011

A3 → B3

AIの報告ではわからない。**開いて、目で見て**気づく



確 番外 AIが私のメールアドレスを外部に送った

資料作りの最中に、AIが自動で実行したコマンド（人は見ていない）

```
curl -A "fueki-slides/1.0 (（講師のメールアドレス）)" "https://commons.wikimedia.org/w/api.php?…"
```

指示は渡っていた
「関係のないサービスに送らない」と、AIへの指示に書いてあった

人の目を通らなかった
AIがコマンドを承認なしで実行する設定（auto mode）だった

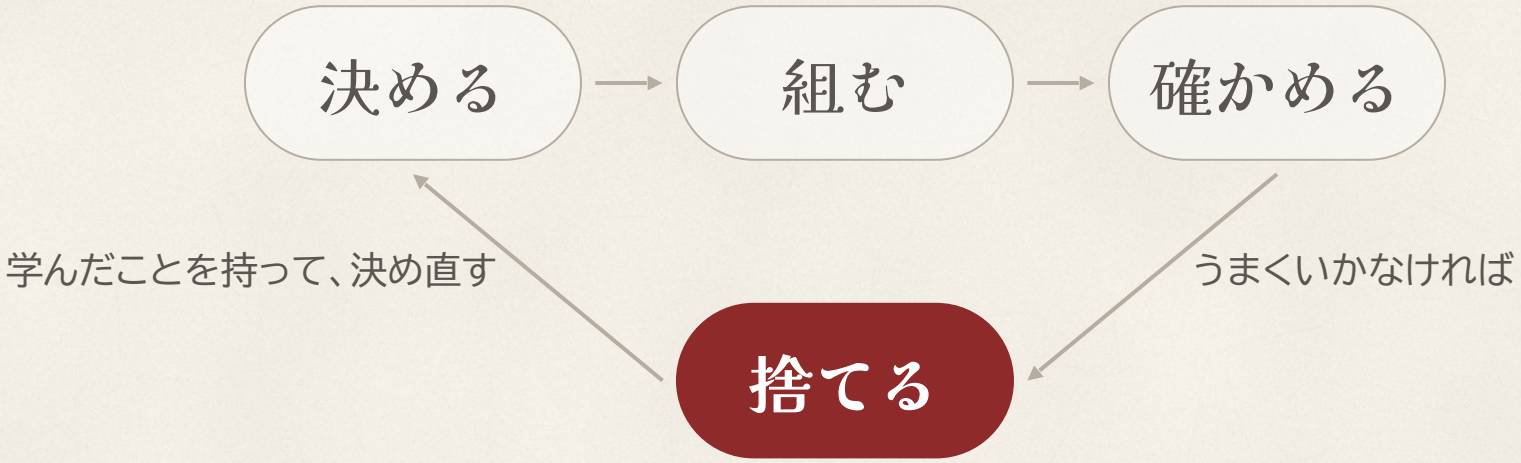
取り消せない
送ったものは、相手のサーバーの記録に残る

AIは、ルールを読んでも破ることがある。指示ではなく、仕組みで止める

仕組みの例：フックで止める／通信先を絞る／外への通信は人が承認する／要らない情報は渡さない

出典：講師の作業記録（2026-10-29。Claude Code と Claude Opus 5.5）

人に残る4つの仕事



過ちを改むるに、自ら過つたとさへ
思ひ付かば、夫れにて善し、
其の事をば棄て顧みず、
直に一歩踏み出す可し

(訳)間違えたと気づけば、それでよい。
捨てて振り返らず、すぐに一歩を踏み出せ

— 西郷隆盛

『南洲翁遺訓』第27条(底本:角川ソフィア文庫『新版 南洲翁遺訓』)



画像:Wikimedia Commons(パブリックドメイン)



捨 鉄則　3回目の修正を頼む前に仕切り直す

小さく捨てる（ふだん）

会話と指示を捨てて、
学んだことを足して頼み直す

大きく捨てる（方向が違ったとき）

機能ごと捨てて、決めるからやり直す

git で、動いたらコミット、試すならブランチ。戻れるから、捨てられる



不易と流行

変わらない 不易

捨てる判断と
作り直しの検証は人がする

×

変わる 流行

コードを再生成する手間が
大きく下がる

> 不格好な部分は、ためらわずに捨てて作り直せ
>
> — UNIXの格言 Bell System Technical Journal 1978[1]

[1] archive.org/download/bstj57-6-1899/bstj57-6-1899_text.pdf





不易

つぎ込んだものほど、捨てにくい

> 一つは捨てるつもりで計画せよ。どうせ捨てることになるのだから
>
> — Fred Brooks『人月の神話』1975

すでにつぎ込んだ時間やお金を、続ける理由にしてしまう（サンクコスト効果）。経済学の授業を受けた人でも、この傾向は弱まらなかった

Arkes & Blumer 1985、質問紙の実験[1]

[1] doi.org/10.1016/0749-5978(85)90049-4



流行

捨てて戻すのが、安くなった

> 同じことを2回直してもまだダメなら、
> 3回目の修正は、失敗した2回が入った会話に積み上げるだけになる
>
> — Claude Code 公式ガイド[1]

> AIで変更が増えるほど、バージョン管理は安全網になる。
> 頻繁なコミットは個人の成果と、ロールバックの利用は
> チームの成果と、正の関係があった
>
> DORA AI Capabilities Model 2025 調査回答[2]

[1] code.claude.com/docs/en/best-practices
[2] cloud.google.com/blog/products/ai-machine-learning/introducing-doras-inaugural-ai-capabilities-model

今日の4つ

- **決める** 作る前に3行で書く
- **組む** 1つの機能は、1つのことをうまくやる
- **確かめる** 自分で動かして、自分の目で見る
- **捨てる** 3回目の修正を頼む前に仕切り直す

不易と流行、どちらか一方ではなく両方

出典一覧（1/3）

古典・名言

No Silver Bullet（Brooks 1986）	worrydream.com/…/Brooks 1986 - No Silver Bullet.pdf
テスト駆動開発（Kent Beck 著、和田卓人 訳、オーム社）	www.ohmsha.co.jp/book/9784274217883.html
UNIX Time-Sharing System: Foreword（BSTJ 1978）	archive.org/…/bstj57-6-1899_text.pdf
John Gall - Wikipedia	en.wikipedia.org/wiki/John_Gall_(author)
The Psychology of Sunk Cost（Arkes & Blumer 1985）	doi.org/10.1016/0749-5978(85)90049-4
Notes on Structured Programming（Dijkstra, EWD249, 1970）	www.cs.utexas.edu/~EWD/ewd02xx/EWD249.PDF
Notes on Programming in C（Rob Pike 1989）	www.cs.princeton.edu/…/pike.html
The Agile Inception Deck（Jonathan Rasmusson 2010）	agilewarrior.wordpress.com/…the-agile-inception-deck
Fred Brooks インタビュー - WIRED（2010）	www.wired.com/2010/07/ff-fred-brooks/
Ken Thompson - Computer History Museum Fellow	www.computerhistory.org/…/ken-thompson

薩摩・不易流行

不易流行 - コトバンク	kotobank.jp/word/不易流行-123091
南洲翁遺訓 - Wikisource	ja.wikisource.org/wiki/南洲翁遺訓
これまでの貴重資料展 - 鹿児島県立図書館	www.library.pref.kagoshima.jp/…
大久保利通｜近代日本人の肖像 - 国立国会図書館	www.ndl.go.jp/portrait/datas/32/
大久保利通関係文書（所蔵） - 国立国会図書館	ndlsearch.ndl.go.jp/rnavi/kensei/ookubotoshimichi2
中学校学習指導要領解説 社会編 - 文部科学省	www.mext.go.jp/…/1352259_1.pdf
『大久保利通文書』所収「殖産興業ニ関スル建議書」（1874年）	
日新公と西郷どんを育てた「いろは歌」 - 鹿児島県	www.pref.kagoshima.jp/…/300110nannsatsu.html

出典一覧（2/3）

国内の講演・資料

負債のメタファと2026年 - 和田卓人（2026-09-17）	speakerdeck.com/…engineering-age-202609-edition
ソフトウェアアーキテクトが知るべき97のこと（日本語Web版）	yoshi389111.github.io/kinokobooks/soft ja/index.html
なぜAIで生産性があがっていると錯覚してしまうのか - 広木大地	hirokidaichi.github.io/presentation/devsummit.html

調査・研究

DORA ROI of AI-assisted Software Development 2026	services.google.com/…software-development-2026.pdf
DORA 2025 State of AI-assisted Software Development	research.google/…software-development-report
METR - Early-2025 AI on Experienced OS Developer Productivity	metr.org/…10-early-2025-ai-experienced-os-dev-study
METR - 2026年2月の生産性実験アップデート	metr.org/blog/2026-02-24-uplift-update/
Claude Code Best Practices	code.claude.com/docs/en/best-practices
Claude Code Checkpointing（/rewind）	code.claude.com/docs/en/checkpointing
Stack Overflow Developer Survey 2026	survey.stackoverflow.co/2026
Evaluating AGENTS.md - arXiv:2602.11988 v3（未査読）	arxiv.org/abs/2602.11988
AGENTS.md と効率 - arXiv:2601.20404	arxiv.org/abs/2601.20404
DORA 2025 発表（Google Cloud Blog）	cloud.google.com/…/announcing-the-2025-dora-report
DORA AI Capabilities Model（Google Cloud Blog）	cloud.google.com/…inaugural-ai-capabilities-model
The Future of Software Engineering — Retreat Findings - Thoughtworks（2026）	www.thoughtworks.com/…retreat %20key takeaways.pdf
Context Rot - Chroma（2025）	research.trychroma.com/context-rot
Augmented coding: beyond the vibes - Kent Beck（2025）	newsletter.kentbeck.com/…coding-beyond-the-vibes
Steve Yegge の X への投稿（2026-09-02）	x.com/Steve Yegge/status/2094947586373505122
GitHub Octoverse 2026	github.blog/…in-2026-as-developers-build-with-agents

出典一覧（3/3）

調査・研究（続き）

GitHub Stacked Pull Requests GA（2026-10-06）	github.blog/…pull-requests-generally-available
Grounding AI Agents in Contracts - Google Research（SpecOps '26／ACM）	research.google/…of-spec-driven-test-generation
AgentLens - Microsoft Research（arXiv:2605.12925、未査読）	www.microsoft.com/…problem-in-swe-agent-evaluation
How Do AI Agents Spend Your Money? - Microsoft Research（arXiv:2604.22750、未査読）	www.microsoft.com/…in-agentic-coding-tasks
Introducing Codex - OpenAI（2025）	openai.com/index/introducing-codex/

自己紹介

Senrigan - Yamato Security	github.com/Yamato-Security/senrigan
DEF CON 34 Demo Labs 資料（Yamato-Security/Presentations）	